Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251188**
Nama Lengkap	**ANGELA REVALINE SETIAWAN**
Minggu ke / Materi	**09 / Pengolahan String dan Regular Expression**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Pengantar String

- String adalah gabungan karakter-karakter yang menjadi satu kesatuan dan berfungsi untuk menyimpan kalimat, baik panjang maupun pendek.
- String adalah jenis data yang dapat menyimpan huruf / karakter.
- String disimpan dalam kode ASCII.
- Tidak semua bahasa pemrograman memiliki tipe data String, contohnya bahasa C.
- String bukan jenis tipe data dasar, karena tipe data ini menyimpan lebih dari satu nilai tunggal sebagai satu kesatuan.
- String juga dapat disebut sebagai kumpulan tipe data karakter / array of character / list of character.

```
nama_lengkap = "Angela Revaline Setiawan"
nama_panggilan = "Valine"
fakultas = "Teknologi Informasi"
program_studi = "Informatika"
```

Gambar 1.1 : Contoh String

2. Pengaksesan String dan Manipulasi String

```
namaku = "Angela Revaline Setiawan"
temanku1 = "Alicia Luna Santoso"
temanku2 = "Cheila Zefanya Wibowo"
temanku3 = "Virjie Gracia Tumimomor"

print(temanku2)     #'Cheila Zefanya Wibowo'
print(namaku[3])    #'e'
print(temanku1[2])  #'i'
print(temanku3[0])  #'V'
```

Gambar 2.1 : Bentuk string secara sederhana.

String pertama kali dibuat dengan deklarasi variabel dan langsung diisi dengan data, string dapat diakses sebagai satu kesatuan dengan menyebut nama variabelnya atau per huruf dengan menyebutkan indeksnya. Indeks dalam string dimulai dari 0 seperti pada list. Indeks harus berupa bilangan bulat, bukan pecahan.

3. Operator dan Metode String

a) OPERATOR in = untuk memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain.

```
kalimat = "Universitas Kristen Duta Wacana di Yogyakarta"
data = "Kristen"
print(data in kalimat)          #True
print("Yogyakarta" in kalimat)  #True
print("Muntilan" in kalimat)    #False
```

Gambar 3.1 : Contoh penggunaan **operator in**

Selain menggunakan operator in, pada String juga bisa dilakukan perbandingan (comparison) yang akan menghasilkan True atau False.

```python
if "valine" > "karin":
    print("Yes")    #Ya
else:
    print("No")     #Tidak

if "satu" == "satu":
    print("Couple") #Sama
```

Gambar 3.2 : Contoh perbandingan dalam string.

b) FUNGSI len = untuk menghitung berapa panjang (jumlah karakter) dari sebuah string.
   - ➢ Operator len(<string>)
   - ➢ Operator len(<string> - 1) = untuk huruf terakhir

```python
kalimat = "Universitas Kristen Duta Wacana di Yogyakarta"
print(len(kalimat))     #output 45

terakhir = kalimat[len(kalimat) -1]
print(terakhir)         #output 'a'

terakhir_indeks = kalimat[-1]
print(terakhir_indeks)  #output 'a'

terakhir_kedua = kalimat[-2]
print(terakhir_kedua)   #output 't'
```

Gambar 3.3 : Contoh penggunaan **fungsi len**

c) TRAVERSING STRING = menampilkan string dengan cara ditampilkan huruf demi huruf menggunakan loop (for dan while).
   - FOR

```python
kalimat = "malam ini"
for i in kalimat:
    print(i, end=' ')
```

Gambar 3.4 : Contoh penggunaan **for** dalam **traversing string**

   - WHILE

```python
kalimat = "malam ini"
i = 0
while i < len(kalimat):
    print(kalimat[i], end=' ')
    i += 1
```

Gambar 3.5 : Contoh penggunaan **while** dalam **traversing string**

d) STRING SLICE
   - ➢ Untuk menampilkan substring pada sebuah string dengan menggunakan indeks dari awal tertentu sampai akhir - 1 tertentu.
   - ➢ Sintaksnya <string>[awal:akhir]
   - ➢ Bagian awal atau akhir boleh dikosongkan

- Perlu diingat bahwa string bersifat immutable, yakni data tidak bisa diubah saat program sedang berjalan

```
kalimat = "malam ini"
awal = 0
akhir = 5

print(kalimat[awal:akhir])      #malam
print(kalimat[3:len(kalimat)])  #am ini
print(kalimat[:6])              #malam
print(kalimat[6:])              #ini
print(kalimat[:])               #malam ini
```

Gambar 3.6 : Contoh penggunaan **string slice**

- Method String

Nama Method	**Kegunaan**	**Penggunaan**
capitalize()	Untuk mengubah string menjadi huruf besar/ kapital	string.capitalize()
count()	Untuk menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	Untuk mengetahui apakah suatu string diakhiri dengan string yang diinputkan	string.endswith()
startswith()	Untuk mengetahui apakah suatu string diawali dengan string yang diinputkan	string.startswith()
find()	Untuk mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() and isupper()	Untuk mengembalikan nilai True jika huruf kecil / huruf besar	string.islower() and string.isupper()
isdigit()	Untuk mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	Untuk menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	Untuk memecah string menjadi token-token berdasarkan pemisah (ex : spasi)	string.split()

e) Operator * dan + pada String

- Operator * = untuk mengalikan bilangan untuk menampilkan string sejumlah perkaliannya.
- Operator + = untuk menggabungkan dua buah string

```
kata1 = "Universitas Kristen"
kata2 = "Duta Wacana "
kata3 = kata1 + " " + kata2

print(kata3)    #hasil adalah penggabungan: Universitas Kristen Duta
print(kata2 * 2)    #hasil adalah Duta Wacana Duta Wacana
```

4. Parsing String

Parsing String adalah langkah-langkah menelusuri string bagian demi bagian untuk mendapatkan / menemukan / mengubah bagian string yang diinginkan.

```
kalimat = "Saya lahir pada tanggal 23-07-2007 di Magelang"
hasil = kalimat.split(" ")

for i in hasil:
    if i[0].isdigit():
        hasil2 = i.split("-")
        print(hasil2[1] + "/" + hasil2[0] + "/" + hasil2[2])

#ouput yang dihasilkan = 07/23/2007
```

Gambar 4.1 : Contoh program yang mengubah format tanggal dalam sebuah kalimat.

5. Pengantar Regex (Regular Expression)

Regex adalah ekspresi pola yang berbentuk kumpulan karakter yang berfungsi untuk menemukan pola / pattern yang sama dengan pola regex di dalam string lain yang ingin dicari.

Regex dapat membantu kita untuk pencarian string dengan pola tertentu, mengganti string dengan pola tertentu, dan menghapus string dengan pola tertentu. Lalu dengan regex kita juga bisa melakukan parsing string yang selama ini biasanya menggunakan perintah split() dan find() saja.

6. Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	“[a-zA-Z]”	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti

			dengan karakter apapun, misal “sayang” akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Diakhiri dengan	this$	Diakhiri dengan this
*	0 s/d tak terhingga karakter	\{}d*	Ada digit minimal 0 maksimal tak terhingga
?	Ada atau tidak (opsional)	\{}d?	Boleh atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(saya|kamu)	Saya atau kamu sebagai satu kesatuan
|	atau	\{}d\{}s	1 digit atau 1 spasi

Tabel 6.1 : Special Character pada Python

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	“R\bin” “Rain\b”
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan_)	\w

\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	“\AThe”
\Z	Digunakan untuk mengetahui apakah sebuah karakter adalah berada di bagian akhir dari kalimat	“End\z”

Tabel 6.2 : Escaped Character pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b, m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari 2 huruf : 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari 1 huruf a-Z

Tabel 6.3 : Himpunan Karakter pada Regex

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
def cek_anagram(a, b):
    a = a.strip().lower()
    b = b.strip().lower()

    if len(a) == len(b):
        return sorted(a) == sorted(b)
    return False

kata1 = str(input("Masukkan kata pertama = "))
kata2 = str(input("Masukkan kata kedua = "))

if cek_anagram(kata1, kata2):
    print(f"'{kata1}' anagram dengan '{kata2}'")
else:
    print(f"'{kata1}' dan '{kata2}' bukan anagram")
```

Penjelasan :

1. Program ini berguna untuk mendeteksi apakah suatu kata adalah anagram dari kata lainnya atau bukan.
2. Membuat fungsi bernama cek_anagram dengan parameter a dan b.
3. Alur kerja program ini adalah membersihkan input dari spasi dengan menggunakan **strip()**.
4. Kemudian mengubah input menjadi huruf kecil semua dengan menggunakan **lower()** untuk menghindari case sensitive.

5. Program akan mengecek apakah panjang kedua kata sama atau berbeda. Jika sama, maka program akan membandingkan huruf pada kata atau kalimat pertama dan kedua menggunakan **sorted()**.
6. Jika berbeda, maka program akan mengembalikan nilai False.
7. Jika huruf antara kata atau kalimat pertama dan kedua sama, maka program akan mengembalikan nilai True (anagram) dan jika huruf berbeda, maka program akan mengembalikan nilai False (bukan anagram).

Output :

```
Masukkan kata pertama = kimi
Masukkan kata kedua = miik
'kimi' anagram dengan 'miik'
```

```
Masukkan kata pertama = halo
Masukkan kata kedua = hello
'halo' dan 'hello' bukan anagram
```

SOAL 2

Source Code :

```
def hitung_kata(a, b):
    a = a.lower()
    b = b.lower()

    kalimat_polos = ''
    for i in a:
        if i.isalpha() or i == ' ':
            kalimat_polos += i

    pisah = kalimat_polos.split()
    jumlah = pisah.count(b)
    return jumlah
```

```
kalimat = str(input("Masukkan kalimatmu : "))
cari_kata = str(input("Masukkan kata yang dicari : "))

jumlah = hitung_kata(kalimat, cari_kata)
print(f"{cari_kata} ada {jumlah} buah")
```

Penjelasan :

1. Program ini berguna untuk menghitung berapa kali suatu kata muncul (frekuensi) dalam suatu kalimat atau teks (bersifat case sensitive).
2. Membuat fungsi bernama hitung_kata dengan parameter a dan b.
3. Alur kerja program ini mengubah input menjadi huruf kecil semua dengan menggunakan **lower()** untuk menghindari case sensitive.
4. Menggunakan perulangan (**for**), program akan mengecek setiap karakter dalam kalimat yang diinput pengguna. Jika karakter bukan huruf atau spasi maka akan dibuang.
5. Kemudian program akan memecah kalimat berdasarkan spasi menggunakan fungsi **split()**.
6. Lalu program akan menghitung berapa kali suatu kata muncul dengan menggunakan fungsi **count()**.

Output :

```
Masukkan kalimatmu : Saya mau makan. Makan itu wajib. Mau siang at
au malam saya wajib makan
Masukkan kata yang dicari : makan
makan ada 3 buah
```

SOAL 3

Source Code :

```
def hapusspasi(kalimat):
    kata = kalimat.split()
    akhir = ' '.join(kata)
    return akhir

print(hapusspasi('saya tidak suka memancing ikan '))
```

Penjelasan :

1. Program ini berguna untuk menghapus spasi berlebih pada kalimat.
2. Membuat fungsi bernama hapusspasi dengan parameter kalimat.
3. Program akan memisah kalimat menjadi list kata dengan menggunakan fungsi **split()**.
4. Lalu menggunakan fungsi **' '.join()** untuk menggabungkan kembali kata dengan satu spasi saja.
5. Lalu fungsi **return** untuk mengembalikan nilai.

Output :

```
saya tidak suka memancing ikan
```

SOAL 4

Source Code :

```
kalimat = str(input("Masukkan kalimatmu : "))
kata = kalimat.split()

kata_terpendek = kata[0]
kata_terpanjang = kata[0]

for i in kata:
    if len(i) < len(kata_terpendek):
        kata_terpendek = i
    if len(i) > len(kata_terpanjang):
        kata_terpanjang = i

print(f"Kata terpendek = {kata_terpendek} dan kata terpanjang
= {kata_terpanjang}")
```

Penjelasan :

1. Membuat variabel bernama kalimat untuk menyimpan input pengguna.
2. Membuat variabel bernama kata untuk menyimpan hasil list kata dari kalimat yang telah menggunakan fungsi **split()**.
3. Membuat variabel kata terpendek dan kata terpanjang untuk menyimpan nilai awal yang akan digunakan sebagai pembanding.
4. Dalam perulangan **for**, jika jumlah huruf pada kata lebih pendek dari kata terpendek maka kata tersebut menjadi kata terpendek baru.
5. Jika jumlah huruf pada kata lebih panjang dari kata terpanjang maka kata tersebut menjadi kata terpanjang baru.
6. Menggunakan fungsi print untuk mengecek testcase.

Output :

```
Masukkan kalimatmu : red snakes and a black frog in the pool
Kata terpendek = a dan kata terpanjang = snakes
```

SOAL 5

Source Code :

```
import re
from datetime import datetime

kalimat = input("Masukkan kalimatmu : ")

daftar_tanggal = re.findall(r"\d{4}-\d{2}-\d{2}", kalimat)
hari_ini = datetime.now()

for tgl in daftar_tanggal:
    tgl_obj = datetime.strptime(tgl, "%Y-%m-%d")
    selisih = (hari_ini- tgl_obj).days

    print(f"{tgl_obj} selisih {selisih} hari")
```

Penjelasan :

1. Program ini berguna untuk mencari tanggal dalam format thn-bln-tgl didalam kalimat, lalu menghitung selisih hari dari tanggal tersebut hingga hari ini.
2. Mengimport library regex yang akan digunakan, (**import re)** dan **from datetime import datetime** untuk mencari pola teks regex dan datetime untuk mengolah tanggal dan waktu.
3. Membuat variabel bernama kalimat untuk menyimpan input pengguna.
4. Mencari semua tanggal dengan Regex dengan menggunakan **re.findall(r"\d-\d\d")**
5. Membuat variabel bernama hari_ini untuk menyimpan data tanggal dan waktu saat program dijalankan dengan menggunakan **datetime.now()**.
6. Perulangan for untuk mengecek setiap tanggal yang ada, lalu mengubah tanggal menjadi string agar dapat menghitung selisih hari dari input menggunakan **datetime.strptime()**.
7. Lalu menghitung selisih dari tanggal hari ini sampai input dan mengambil jumlah harinya saja menggunakan **.days**.
8. Gunakan fungsi **print** untuk mengeluarkan output yang diinginkan.

Output :

```
Masukkan kalimatmu : Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki
beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura
 TL: 1783-06-08) dan Ki Hajar Dewantara (TL: 1889-05-02).
1945-08-17 00:00:00 selisih 29463 hari
1785-11-11 00:00:00 selisih 87815 hari
1783-06-08 00:00:00 selisih 88702 hari
1889-05-02 00:00:00 selisih 50023 hari
```

SOAL 6

Source Code :

```
import re

import random

import string


kalimat = input("Masukkan kalimatmu = \n")


for i in re.findall(r"\S+@\S+", kalimat):

    username = i.split("@")[0]

    password = ''.join(random.choices(string.ascii_letters +
string.digits, k = 8))
```

```
print(f"{i} username: {username}, password: {password}")
```

Penjelasan :

1. Program ini berguna untuk mencari email dalam kalimat lalu diambil username dari sebelum @ dan membuat password acak 8 karakter.
2. Import library yang diperlukan dalam program ini yaitu **re, random, dan string.**
3. Membuat variabel bernama kalimat untuk menyimpan input pengguna.
4. Mencari semua email yang ada dalam kalimat menggunakan perulangan for.
5. Lalu program akan mengambil username dari email menggunakan fungsi **split()** dan disimpan dalam variabel username.
6. Membuat password acak 8 karakter menggunakan fungsi **‘ ‘.join(random.choices(string.ascii_letters + string.digits, k = 9)).**
7. Gunakan fungsi **print** untuk mengeluarkan output yang diinginkan.

Output :

```
Masukkan kalimatmu =
valine@gmail.com dimiliki oleh valine. cei@mail.com dimiliki oleh cei. cia@students
.ukdw.ac.id dimiliki oleh cia. vi@ti.ukdw.ac.id dimiliki oleh vi.
valine@gmail.com username: valine, password: G4VxsvDM
cei@mail.com username: cei, password: HVsoN9TN
cia@students.ukdw.ac.id username: cia, password: vo6Sr5dV
vi@ti.ukdw.ac.id username: vi, password: qFlrhFZG
```